

2026



MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Plan de test



*Développement sous la
direction du groupe L2F2*

Direction : David JANISKEK

Plan de test

Projet L2F2 – Déquantification

Identification du document		Éléments de vérification du document	
Référence	<i>PDT-L2F2</i>	Validé par :	<i>Gaël MAHE</i>
Date	<i>19/05/26</i>	Validé le :	<i>19/05/26</i>
Auteurs	<i>KANGNI Dina et, ACHAK Salim</i>	Soumis le :	<i>19/05/26</i>
Version du document		Confidentialité :	Projet académique, réservé aux étudiants UFR Maths-Info de l'université Paris Descartes

Élément d'authentification

	Maitrise d'ouvrage	<i>Gaël MAHE et David JANISZEK</i>	
	Maitrise d'œuvre	<i>Equipe L2F2</i>	
	Chef de projet	Abdallah BENALI	

Mots clés : *Julia, test d'intégration, test unitaire, déquantification, sous-quantification*

SOMMAIRE

1.	Introduction	5
1.1.	Objectifs	5
1.2.	Documents de référence	5
2.	Guide de lecture	6
2.1.	Maitrise d'œuvre	6
2.2.	Maitrise d'ouvrage	6
3.	Concept de base	6
4.	Tests fonctionnels	7
	TF-01 – Programme auxiliaire (module <i>construction.jl</i>) : génération de xQ et P	8
	TF-02 – Programme principal (module déquantification) : déquantification	9
	TF-03 – Interface graphique module (interface.jl)	9
5.	Tests d'intégration	10
	TI-01 : construction.jl vers dequantification.jl	10
	TI-02 : déquantification.jl vers interface.jl	11
6.	Tests unitaires	12
6.1.	Module arbre.jl	12
6.2.	Module construction.jl	13
6.3.	Module dequantification.jl	14
7.	Vérification de la documentation	15
8.	Glossaire	15
9.	Reference	16
10.	Index	16

1. Introduction

Ce document présente le plan de test du projet L2F2, composé de ACHAK Salim, KANGNI Dina et BENALI Abdallah. L'objectif est de s'assurer que les modules développés en Julia fonctionnent correctement et que le jeu de données entré par l'utilisateur, peut bien être reconstruit à partir de sa version sous-quantifiée xQ et de son histogramme P.

Il est entièrement rédigé par KANGNI Dina et ACHAK Salim sous la direction de Gaël MAHE qui en assure la relecture finale et l'approuve.

1.1. Objectifs

Le projet implémente un algorithme de déquantification à 1 bit. A partir d'un jeu de données, il génère sa version sous-quantifiée ainsi que son histogramme.

Les tests sont effectués à l'aide du module *test.jl* de Julia.

Chaque module est testé de manière indépendante.

On rédige un ultime test *runtest.jl*, qui regroupe tous les tests et s'assure de leur intégration.

Ces tests ont pour but de vérifier que l'arbre construit ainsi que son parcours soient bien cohérents avec l'histogramme P et les fonctions rédigées.

1.2. Documents de référence

Différents documents ont servi à l'élaboration de ce plan de test. Le cahier des charges nous a permis de déterminer le contexte dans lequel s'inscrit l'élaboration de l'application ainsi que les contraintes établies par la maîtrise d'œuvre.

Le comportement de chaque module (*arbre.jl*, *construction.jl*, *dequantification.jl*, *interface.jl*, *application.jl*) et de chaque fonction a été détaillé dans le cahier de conception général et détaillé qui nous a permis de rédiger ce plan.

2. Guide de lecture

2.1. Maitrise d'œuvre

La maitrise d'œuvre est assurée par la deuxième équipe du projet L2F. Elle est composée de quatre membres : KANGNI Dina, BENALI Abdallah, ACHAK Salim et ZOUAD Mohammed.

Ce document lui permet de vérifier que toutes les fonctions et modules développés correspondent bien au cahier de conception générale et détaillée, qu'ils ne dépassent pas le cadre qui y a été défini et qu'ils respectent les directives de la maitrise d'ouvrage.

Ainsi, la maitrise d'œuvre peut avoir une vue d'ensemble du projet mais aussi identifier les modules et les dépendances.

Ce document est conçu pour être facilement traduisible pour la phase de développement de l'application.

2.2. Maitrise d'ouvrage

La maitrise d'ouvrage est composée de l'encadrant du projet, Gaël MAHE, ainsi que du responsable de l'UE projet, David JANISZEK.

Ce document lui permet de vérifier la conformité technique de l'application avec le cahier des charges et le cahier de recette. Elle s'assure ainsi que ses directives ont été respectées.

3. Concept de base

Ce projet consiste à développer une application de déquantification à 1 bit.

La sous-quantification à 1 bit supprime le bit de poids faible de chaque valeur. Ainsi, soit x , le jeu de données initial.

Pour tout $x[n]$, la version sous-quantifiée $xQ[n]$ est le nombre pair inférieur.

Cette opération est irréversible.

Par conséquent, on a :

$x[n] = xQ[n]$ ou $x[n] = xQ[n] + 1$.

Soit l'histogramme P.

Cet histogramme représente tous les couples de valeurs possibles dans x ainsi que leur fréquence dans la série originale. Il nous permet de reconstruire x en partant de xQ.

On l'implémente sous la forme d'un dictionnaire (`Dict{Tuple{Int16,Int16}, Int}`), avec la clé qui représente le couple consécutif courant $((x[n-1], x[n]))$ et la valeur représentant le nombre de fois qu'il apparaît dans la série originale.

On utilise un arbre binaire pour représenter toutes les solutions possibles qu'on peut trouver.

Chaque nœud représente une valeur et possède une copie de P qu'il décrémente dès qu'il forme un couple qui existe dans l'histogramme de son père.

Ainsi le fils transmet à son tour sa copie de P à son fils et récursivement.

Dans le cas où le couple formé ne serait pas compatible avec P, le fils est élagué.

Si son parent devient une feuille, il est élagué et récursivement.

On dit d'une branche qu'elle est une solution si à la fin sa taille est égale à la taille de xQ.

On parcourt l'arbre en profondeur.

4. Tests fonctionnels

Les tests fonctionnels vérifient que nos fonctions et nos modules font bien ce pour quoi ils ont été conçus.

Rappel des modules que nous avons :

- **arbre.jl**

Ce module gère la structure de l'arbre et les fonctions associées.

- **construction.jl**

On l'appelle aussi **programme auxiliaire**. Il se charge de générer xQ et P.

- **dequantification.jl**

On l'appelle aussi **programme principal**. Il se charge d'implémenter l'algorithme de déquantification et de générer toutes les solutions trouvées.

- **Interface.jl**

Il correspond à la modélisation de l'interface graphique.

TF-01 – Programme auxiliaire (module *construction.jl*) : génération de xQ et P

Identification	Description	Contraintes	Dépendances
TF-01	Vérifier que l'application lit correctement un fichier binaire de type x.dat, génère xQ par sous-quantification à 1 bit et l'histogramme P des couples de valeurs successives de x, puis les sauvegarde sur le disque	Julia installé. Fichier x.dat fourni par l'encadrant disponible dans test/data/input/.	Aucune.

Procédure de test

Entrée : x.dat contenant la série temporelle originale encodée en binaire Int16.

Résultat attendu : xQ.dat contenant les valeurs sous-quantifiées correspondantes, P.ppm contenant l'histogramme des couples de valeurs successives de x.

Critères de validation :

- Pour tout n , $xQ[n] = x[n] \& 0xFFFE$, c'est-à-dire $xQ[n] = x[n]$ si $x[n]$ est pair, $xQ[n] = x[n] - 1$ si $x[n]$ est impair.
- Pour tout couple (i, j) , $P[(i,j)]$ est égal au nombre d'occurrences du couple $(x[n-1], x[n]) = (i, j)$ dans x .
- P est construit à partir de x et non de xQ .
- $xQ.dat$ et $P.ppm$ sont bien créés dans le dossier de sauvegarde attendu.

TF-02 – Programme principal (module déquantification) :
déquantification

Identification	Description	Contraintes	Dépendances
TF-02	Vérifier que l'algorithme de déquantification produit au moins une solution à partir de xQ et P, et que les fichiers solution_k.dat sont générés. Il les sauvegarde sur le disque par la suite.	TF-01 doit avoir été exécuté ou un jeu de données xQ/P doit être disponible.	TF-01

Procédure de test

Entrée : xQ = [2, 2], P = {(2,3)=>1}.

Résultat attendu : au moins un fichier solution_1.dat généré contenant [2, 3].

Critère de validation : le fichier existe, contient les bonnes valeurs, une valeur par ligne.

TF-03 – Interface graphique module (interface.jl)

Identification	Description	Contraintes	Dépendances
TF-03	Vérifier que l'interface permet d'importer un fichier, lancer l'algorithme, visualiser l'arbre et exporter les solutions. Que les boutons fonctionnent que les messages d'erreur et de succès s'affichent que le dossier temp où sont sauvegardées soit vidé à chaque début de session que les solutions sont bien	GLMakie, GraphMakie, NativeFileDialog, ZipFile installés. Julia 1.10 recommandé.	TF-01, TF-02

	mises dans une archive zip		
--	-------------------------------	--	--

Procédure de test

On lance l'application avec la commande `julia project src/application.jl`. On importe le fichier `x.dat` via le bouton Importer. On clique sur Lancer et on vérifie que l'arbre s'affiche et que le pourcentage évolue. On exporte ensuite les solutions.

Critère de validation : fenêtre s'ouvre, arbre visible, fichiers exportés dans le dossier choisi.

5. Tests d'intégration

Nous allons vérifier à l'aide de tests d'intégration que les modules s'articulent bien entre eux. C'est-à-dire qu'on s'assure que `xQ` et `P` générés par *construction.jl* sont exploitables par *dequantification.jl* et *interface.jl*. Nous nous appuyons principalement sur des logs.

TI-01 : *construction.jl* vers *dequantification.jl*

Identification

TI-01

Description

On vérifie que les données produites par *construction.jl* sont bien exploitables par *dequantification.jl* sans traitement intermédiaire.

But du test

S'assurer que `xP` et `P` générés par *construction.jl* sont dans un bon format pour être passés directement à `déquantifier()`.

Principe de réalisation

On génère `xQ` et `P` à partir d'un signal `x` connu puis on lance `dequantifier()` avec ces données et on vérifie que le signal original est retrouvé parmi les solutions.

Contraintes

Les tests unitaires de *construction.jl* et *dequantification.jl* doivent passer avant ce test.

Dépendances

TU-CONST-01 a TU-CONST-06, TU-DEQU-01 a TU-DEQU-07

Procédure de test

Entrée : $x = [2, 3, 2, 3]$. On génère xQ et P avec `construction.jl` puis on lance `dequantifier()` avec ces données.

Résultat attendu : au moins un fichier `solution_k.dat` contenant $[2, 3, 2, 3]$.

Critère de validation : le signal original est retrouvé parmi les solutions générées.

Statut : OK

TI-02 : déquantification.jl vers interface.jl

Identification

TI-02

Description

On vérifie que les fonctions de callback passées par `interface.jl` à `dequantifier()` sont bien appelées pendant l'exécution et que les données affichées dans l'interface correspondent à ce que `dequantification.jl` a calculé.

But du test

S'assurer que les callbacks `mis_a_jour_arbre` et `mis_a_jour_branches` passés à `dequantifier()` sont bien appelés pendant l'exécution, et que le nombre de branches affiché dans l'interface correspond au résultat réel de l'algorithme.

Principe de réalisation

On passe à `dequantifier()` des callbacks qui incrémentent des compteurs locaux à chaque appel. On vérifie que `mis_a_jour_arbre` est appelé au moins une fois, que `ajouter_solution` est appelé autant de fois qu'il y a de fichiers `solution` générés, et que `mis_a_jour_branches` reçoit le bon compteur final.

Contraintes

Les tests unitaires de `dequantification.jl` et le test TI-01 doivent passer avant ce test. L'interface n'a pas besoin d'être lancée, on teste uniquement les appels de callbacks.

Dépendances

TI-01

Procédure de test

Entrée : xQ = [2, 2, 2, 2], P = Dict{(Int16(2),Int16(2))=>2}.

On appelle `dequantifier()` avec des callbacks qui comptent leurs appels. On vérifie que `mis_a_jour_arbre` est appelé au moins une fois, que `ajouter_solution` est appelé autant de fois qu'il y a de fichiers générés, et que `mis_a_jour_branches` reçoit le bon compteur.

Critère de validation : compteurs cohérents avec les fichiers solution générés.

Statut : en attente

6. Tests unitaires

On vérifie chacune des fonctions de chaque module à l'aide de tests unitaires.

6.1. Module `arbre.jl`

Les tests sont dans le fichier `test/test_arbre.jl`. On les lance depuis le dossier *branches* avec la commande `julia project test/test_arbre.jl`. Aucune dépendance externe n'est nécessaire.

ID	Fonction testée	Entrée	Résultat attendu	Statut
TU-ARB-01	<code>creer_arbre()</code>	Aucune	racine non vide, nb_branche=0	OK
TU-ARB-02	<code>est_feuille()</code>	Noeud sans enfants	true	OK
TU-ARB-03	<code>est_feuille()</code>	Noeud avec 1 enfant	false	OK
TU-ARB-04	<code>ajouter_enfant!()</code>	parent, valeur=4, pos=false, P	enfant cree, nb_branche=1	OK
TU-ARB-05	<code>supprimer_noeud!()</code>	Supprimer e1 parmi e1 et e2	e2 reste, nb_branche=1	OK

TU-ARB-06	compter_branches()	Arbre avec 2 feuilles	2	OK
TU-ARB-07	extraire_serie()	Feuille e3 avec ancetres e1 e2	[2,3,4] racine exclue	OK

6.2. Module construction.jl

Les tests sont dans le fichier test/test_construction.jl. On les lance depuis le dossier branches avec la commande `julia project test/test_construction.jl`. Les tests créent des fichiers temporaires et les suppriment après exécution.

ID	Fonction testée	Entrée	Résultat attendu	Statut
TU-CONST-01	lire_serie()	Fichier binaire Int16 [2,4,7]	vecteur Int16 [2,4,7]	OK
TU-CONST-02	lire_serie()	Fichier inexistant	Exception levee	OK
TU-CONST-03	sous_quantifier()	[2,3,4,5,6,7]	[2,2,4,4,6,6]	OK
TU-CONST-04	construire_p()	[2,3,2,3]	P (2,3)=2 avec et (3,2)=1	OK
TU-CONST-05	sauvegarder_xq() + lire_serie()	xQ=[2,4,6]	Aller-retour identique	OK

TU-CONST-06	sauvegarder_p())+ lire_p()	P avec 2 entrées	Aller-retour identique	OK
--------------------	--------------------------------	------------------	------------------------	----

6.3. Module dequantification.jl

ID	Fonction testée	Entrée	Résultat attendu	Statut
TU-DEQU-01	est_compatible()	noeud val=2, P={{(2,3)}=>1}	true pour val=3	OK
TU-DEQU-02	est_compatible()	noeud val=2, P={{(2,4)}=>0}	false pour val=4	OK
TU-DEQU-03	sauvegarde_r_solution()	solution=[2, 3,4,5]	Fichier solution_1.dat créé	OK
TU-DEQU-04	elaguer!()	Supprimer feuille, parent devient feuille	Parent aussi élagué	OK
TU-DEQU-05	elaguer!()	Supprimer feuille, parent a encore un frère	Parent conservé	OK
TU-DEQU-06	dequantifie r()	xQ=[2,2], P={{(2,3)}=>1}	Au moins 1 solution	OK

TU-DEQU-07	dequantifie r()	xQ=[2,2], P vide	0 solution	OK
-------------------	--------------------	---------------------	------------	----

7. Vérification de la documentation

Le README.md décrit les prérequis, les étapes d'installation et de lancement via install.bat/run.bat sous Windows et install.sh/run.sh sous Linux et macOS, ainsi que le guide utilisateur et la structure du projet.

Le dossier doc/ contient le cahier des charges, le cahier de recette et le cahier de conception général et détaillé ainsi que ce plan de test

La licence Apache 2.0 est présente à la racine du projet dans LICENSE.txt et mentionne explicitement les auteurs.

Les commentaires dans le code source sont rédigés en français et décrivent le rôle de chaque fonction et structure. La documentation technique peut être générée via Documenter.jl à partir de ces commentaires.

8. Glossaire

xQ :

Signal sous-quantifié à 1 bit.

P :

Désigne l'histogramme des couples de valeurs successives du signal original.

DFS :

(Depth-First Search) est le parcours en profondeur utilisé pour construire l'arbre.

Elagage :

La suppression d'une branche de l'arbre lorsqu'elle est incompatible avec P.

Racine fictive :

Le nœud de valeur 0 servant de point de départ, non inclus dans les solutions.

Int16 :

Type entier signé sur 16 bits utilisé pour toutes les valeurs du signal.

9. Reference

The Julia Language. « Julia 1.12 Documentation ». <https://docs.julialang.org/en/v1.12/>.

KANGNI, D., BENALI, A., & ACHAK, S. (2026). *Cahier des charges L2F2*.

KANGNI, D., BENALI, A. & ACHAK, S. (2026). *Cahier de conception générale et détaillée L2F2*.

« Unit Testing · The Julia Language ». <https://docs.julialang.org/en/v1/stdlib/Test/>.

10. Index

arbre.jl, 4, 12

construction.jl, 4, 13

dequantification.jl, 4, 11

déquantification.jl, 4, 11

GLMakie, 9

GraphMakie, 9

'histogramme', 15

Julia, 5, 8, 9, 16

NativeFileDialog, 9

Tests d'intégration, 4, 10

Tests fonctionnels, 4, 7

Tests unitaires, 4, 12

xQ, 4, 5, 6, 7, 8, 9, 10, 11, 13, 14, 15

ZipFile, 9